

PRAKTIKUM BASIS DATA

MODUL 9

09_Scripting_Procedure_dan_Function



**Universitas
Telkom**

Oleh:

Surya Pradipta

103122400061

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

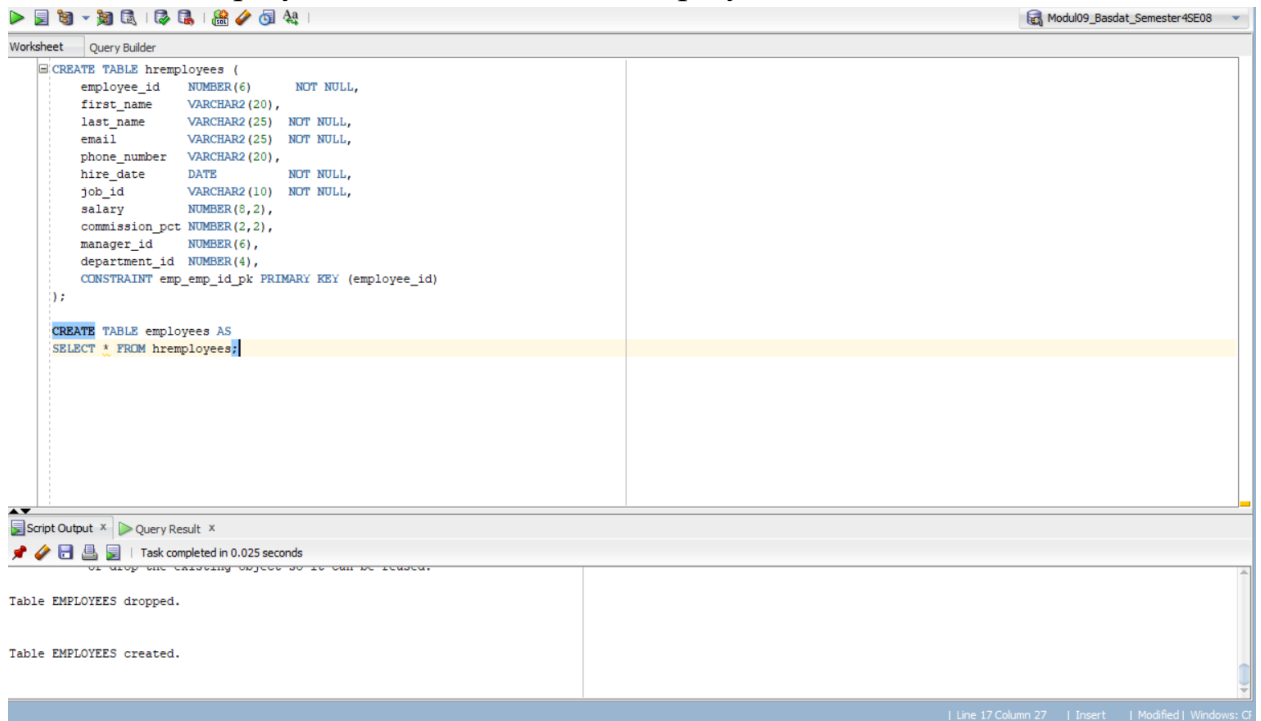
DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. create table employees as select * from hremployees.



The screenshot shows the SQL Developer interface with the 'Query Builder' tab active. The main editor contains the following SQL code:

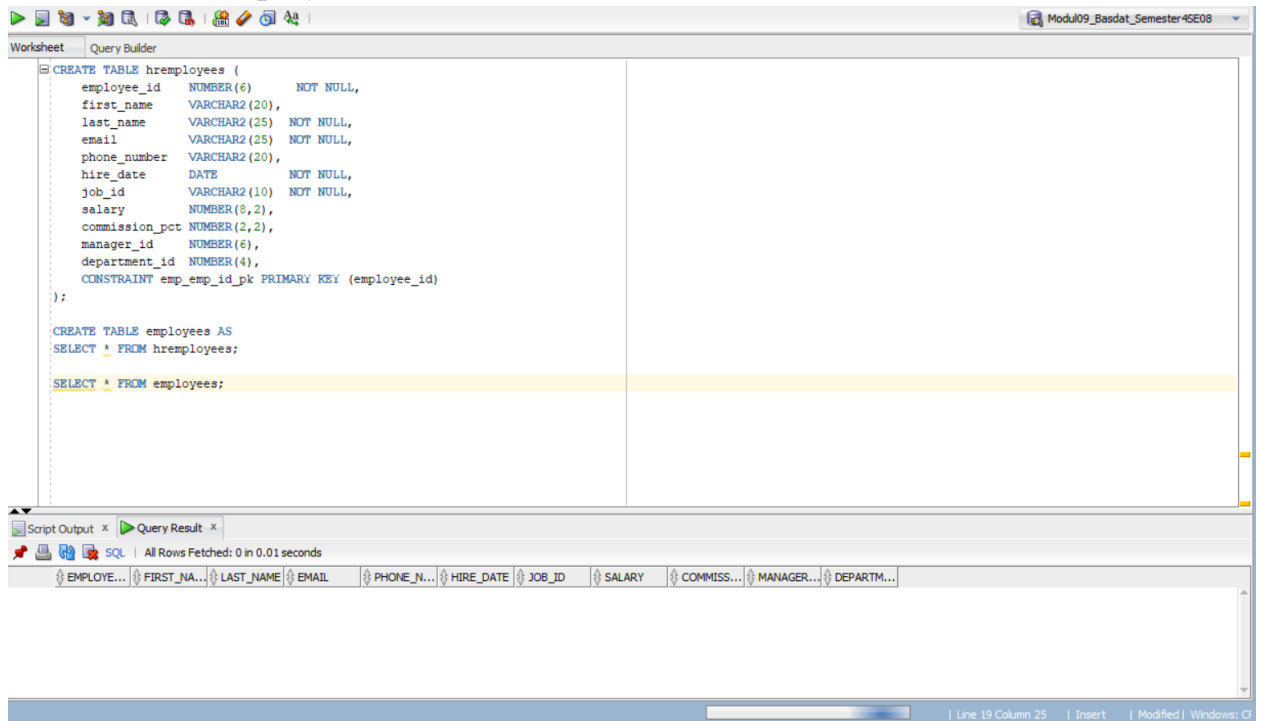
```
CREATE TABLE hremployees (  
  employee_id NUMBER(6) NOT NULL,  
  first_name VARCHAR2(20),  
  last_name VARCHAR2(25) NOT NULL,  
  email VARCHAR2(25) NOT NULL,  
  phone_number VARCHAR2(20),  
  hire_date DATE NOT NULL,  
  job_id VARCHAR2(10) NOT NULL,  
  salary NUMBER(8,2),  
  commission_pct NUMBER(2,2),  
  manager_id NUMBER(6),  
  department_id NUMBER(4),  
  CONSTRAINT emp_emp_id_pk PRIMARY KEY (employee_id)  
);  
  
CREATE TABLE employees AS  
SELECT * FROM hremployees;
```

The 'Script Output' tab at the bottom shows the execution results:

```
Table EMPLOYEES dropped.  
  
Table EMPLOYEES created.
```

The status bar at the bottom indicates 'Line 17 Column 27 | Insert | Modified | Windows: C'.

2. select * from employees.



The screenshot shows the SQL Developer interface with the 'Query Builder' tab active. The main editor contains the following SQL code:

```
CREATE TABLE hremployees (  
  employee_id NUMBER(6) NOT NULL,  
  first_name VARCHAR2(20),  
  last_name VARCHAR2(25) NOT NULL,  
  email VARCHAR2(25) NOT NULL,  
  phone_number VARCHAR2(20),  
  hire_date DATE NOT NULL,  
  job_id VARCHAR2(10) NOT NULL,  
  salary NUMBER(8,2),  
  commission_pct NUMBER(2,2),  
  manager_id NUMBER(6),  
  department_id NUMBER(4),  
  CONSTRAINT emp_emp_id_pk PRIMARY KEY (employee_id)  
);  
  
CREATE TABLE employees AS  
SELECT * FROM hremployees;  
  
SELECT * FROM employees;
```

The 'Query Result' tab at the bottom shows the execution results:

```
All Rows Fetched: 0 in 0.01 seconds
```

EMPLOYEE...	FIRST_NA...	LAST_NAME	EMAIL	PHONE_N...	HIRE_DATE	JOB_ID	SALARY	COMMISS...	MANAGER...	DEPARTM...
-------------	-------------	-----------	-------	------------	-----------	--------	--------	------------	------------	------------

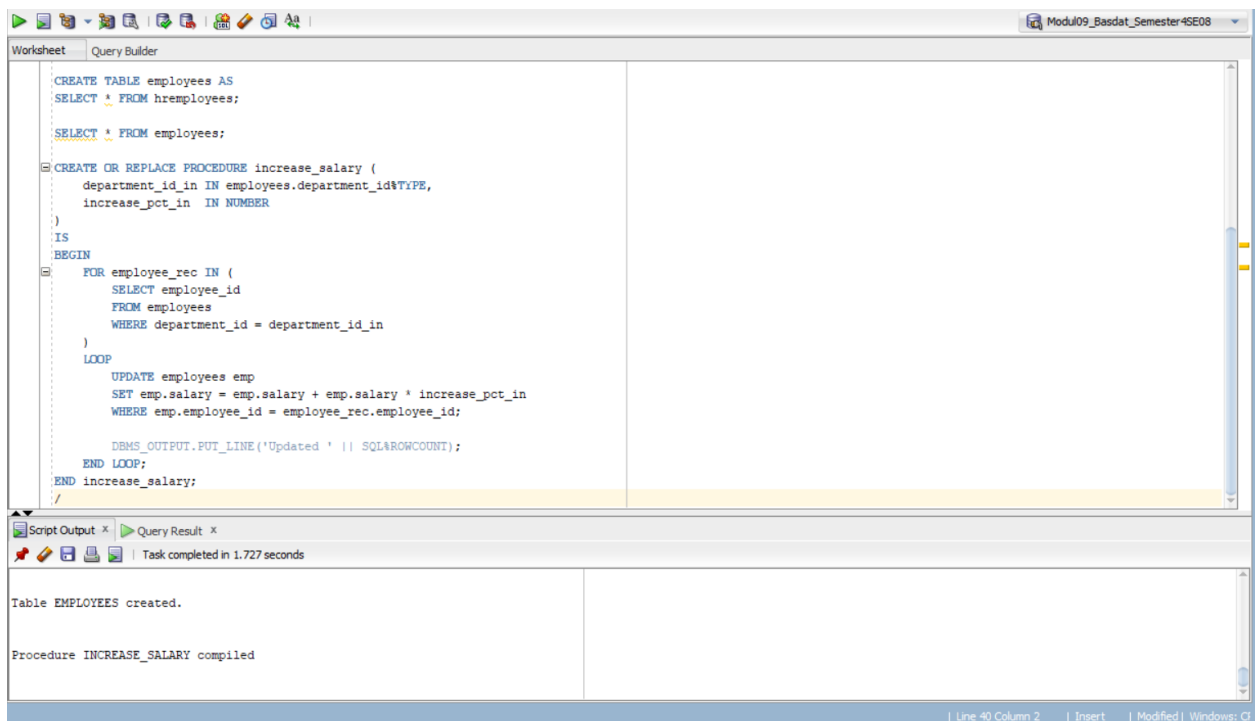
The status bar at the bottom indicates 'Line 19 Column 25 | Insert | Modified | Windows: C'.

```

1 CREATE OR REPLACE PROCEDURE increase_salary (
2 department_id_in IN employees.department_id%TYPE,
3 increase_pct_in IN NUMBER)
4 IS
5 BEGIN
6     FOR employee_rec
7         IN (SELECT employee_id
8             FROM employees
9             WHERE department_id = increase_salary.department_id_in)
10    LOOP
11        UPDATE employees emp
12            SET emp.salary = emp.salary + emp.salary * increase_salary.increase_pct_in
13            WHERE emp.employee_id = employee_rec.employee_id;
14        DBMS_OUTPUT.PUT_LINE ('Updated ' || SQL%ROWCOUNT);
15    END LOOP;
16 END increase_salary;
17

```

3.



4. BEGIN increase_salary (90, .50); ROLLBACK; END; / select * from employees where department_id = 90

The image shows two screenshots of the SQL Developer interface. The top screenshot displays the execution of a PL/SQL procedure named `INCREASE_SALARY`. The procedure is designed to increase the salary of employees in a specific department by a given percentage. The code includes a `FOR` loop to iterate over employees in department 90 and an `UPDATE` statement to increase their salaries. The procedure is successfully compiled and executed, as indicated by the messages in the Script Output pane.

```
CREATE OR REPLACE PROCEDURE increase_salary (  
    department_id_in IN employees.department_id%TYPE,  
    increase_pct_in IN NUMBER  
)  
IS  
BEGIN  
    FOR employee_rec IN (  
        SELECT employee_id  
        FROM employees  
        WHERE department_id = department_id_in  
    )  
    LOOP  
        UPDATE employees emp  
        SET emp.salary = emp.salary + emp.salary * increase_pct_in  
        WHERE emp.employee_id = employee_rec.employee_id;  
        DBMS_OUTPUT.PUT_LINE('Updated ' || SQL%ROWCOUNT);  
    END LOOP;  
END increase_salary;  
  
BEGIN  
    increase_salary(90, .50);  
    ROLLBACK;  
END;
```

The bottom screenshot shows the same code in the SQL Developer interface, but with the `ROLLBACK;` statement removed. The code is executed, and the results are displayed in the Query Result pane. The results show that the salaries of the three employees in department 90 have been increased by 50%.

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	Lex	De Haan	LDEHAAN	515.123.4569	13-01-2001	AD_VP	17000	(null)	100	90
2	Neena	Kochhar	NKOCHHAR	515.123.4568	21-09-2005	AD_VP	17000	(null)	100	90
3	Steven	King	SKING	515.123.4567	17-06-2003	AD_PRES	24000	(null)	(null)	90

5. Apa yang terjadi jika statement ROLLBACK dihapus pada sql code no 4.
Salary employee di department 90 benar-benar naik 50%